



Algorithms and Data Structures

Sujeong Kim



UNIVERSITY OF
MARYLAND

Dynamic Programming (1)

Fibonacci Numbers

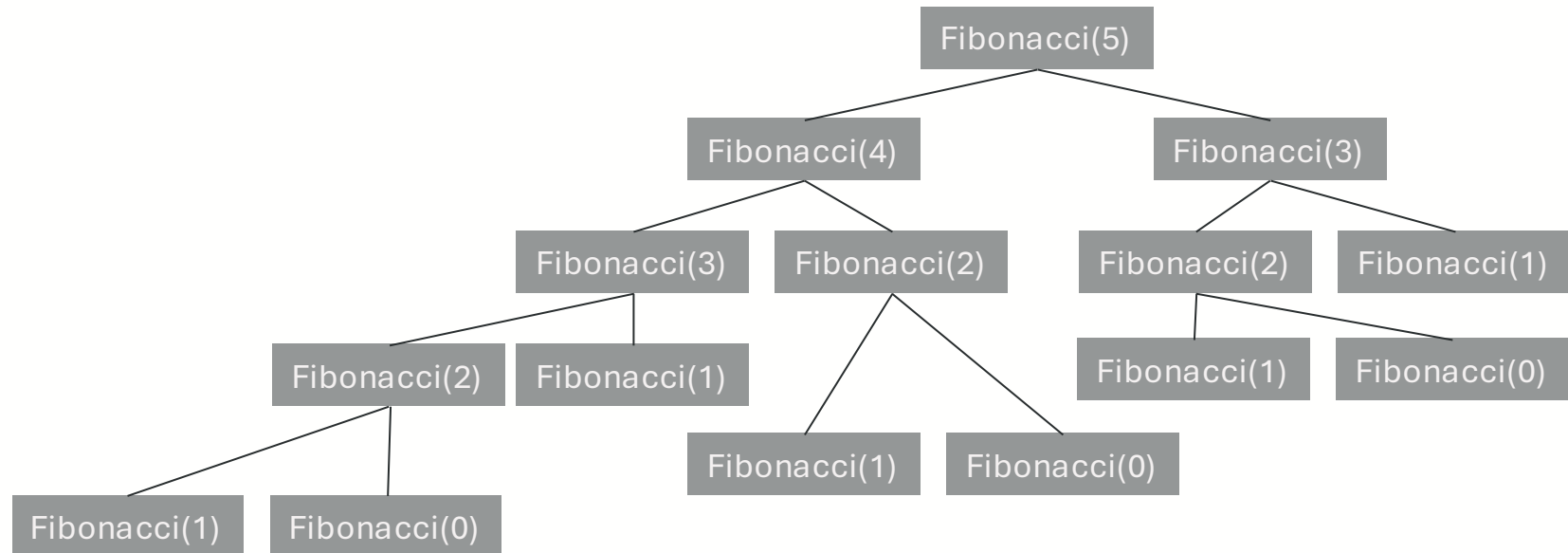
Slide Credit: These slides are adapted from materials by Prof. Manocha, with modifications and additions by Sujeong Kim.

Dynamic Programming (DP)

Powerful technique for solving complex problems by breaking it down into simpler subproblems.

Properties of Problems Solved using DP (1)

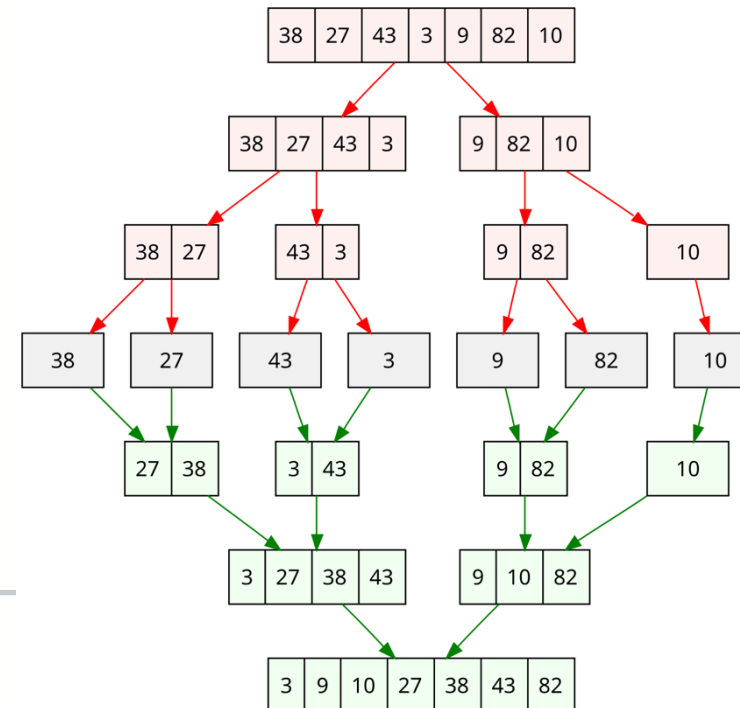
- Overlapping Subproblems:
 - A problem can be broken down into smaller subproblems that are reused multiple times



Properties of Problems Solved using DP (1)

- Overlapping Subproblems:
 - A problem can be broken down into smaller subproblems that are reused multiple times
 - Divide-and-conquer algorithms like Merge Sort typically work on independent (non-overlapping) subproblems

Merge sort
example



Properties of Problems Solved using DP (2)

- Optimal Substructure:
 - A problem is said to have optimal substructure if an optimal solution can be constructed from optimal solutions of its subproblems
 - Recall that greedy algorithms can also be used for optimization problems
 - Local optimal choice at each step
 - Not always guaranteed to find the optimal solution
 - DP guarantees finding the globally optimal solution by considering all subproblems

Properties of Problems Solved using DP (2)

- Optimal Substructure:
 - A problem is said to have optimal substructure if an optimal solution can be constructed from optimal solutions of its subproblems
- Shortest Path Optimal Substructure
 - If a node V lies on the optimal path from Start to Goal, then the path from V to Goal must also be the optimal path between those two points

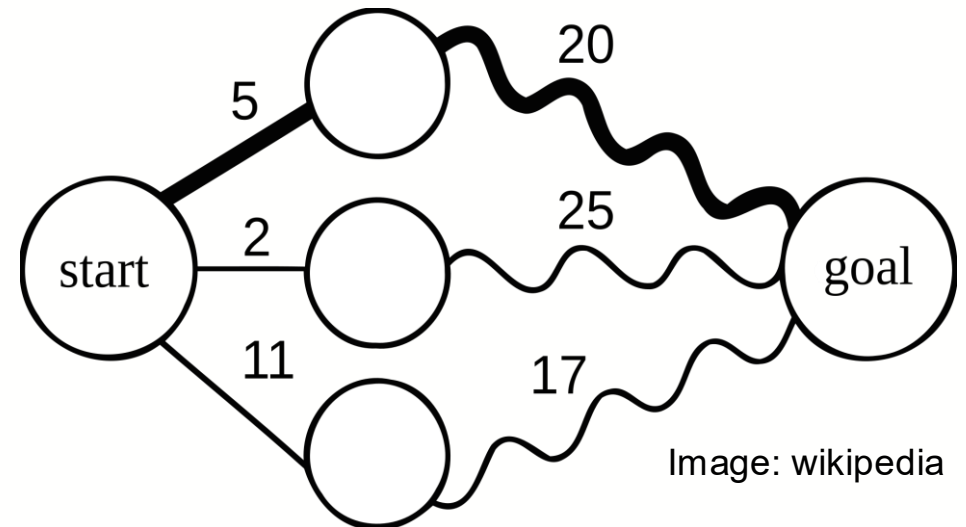


Image: wikipedia

Properties of Problems Solved using DP (2)

- Optimal Substructure:
 - A problem is said to have optimal substructure if an optimal solution can be constructed from optimal solutions of its subproblems
- 0/1 Knapsack Optimal Substructure
- Suppose the absolute best way to fill a 13kg bag include item 3 (which weighs 3kg)
- The remaining items in the bag ([item1, item2]) must be the optimal solution for a smaller Knapsack with 10kg capacity
- If they weren't the best for 10kg, you could replace them with a better set, making your 13kg bag even more valuable

Dynamic Programming (DP)

- Properties of Problems Solved using DP
 - Overlapping Subproblems
 - Optimal Substructure
- DP solves these problems efficiently by computing each subproblem only once and reusing their results
 - Memoization (top-down)
 - Tabulation (bottom-up)
- We will explore these concepts through the Fibonacci sequence problem.

Fibonacci Sequence

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}	F_{18}	F_{19}	...
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1597	2584	4181	...

- We define the Fibonacci numbers by the following recurrence:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_i = F_{i-1} + F_{i-2}, \text{ for } i \geq 2$$

- Given n , calculate F_n

Fibonacci Number

- Straightforward Algorithm
 - Recursion
 - Divide-and-conquer paradigm

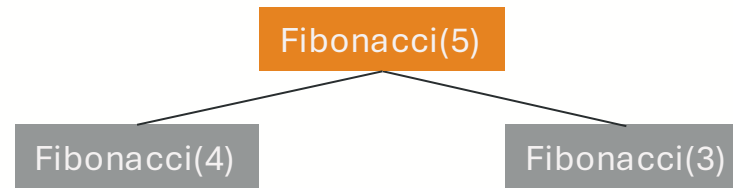
$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_i &= F_{i-1} + F_{i-2}, \text{ for } i \geq 2 \end{aligned}$$

```
function Fibonacci(n)
    if n <= 1
        return n
    else
        return Fibonacci(n-1) + Fibonacci(n-2)
```

Fibonacci Number – Straightforward Method

- Fibonacci(5)

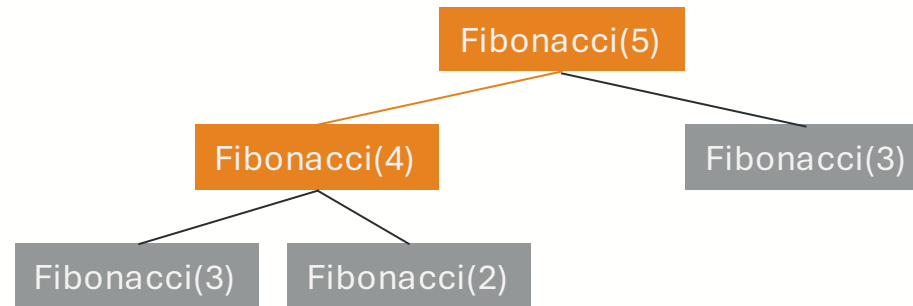
```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Fibonacci Number – Straightforward Method

- Fibonacci(5)

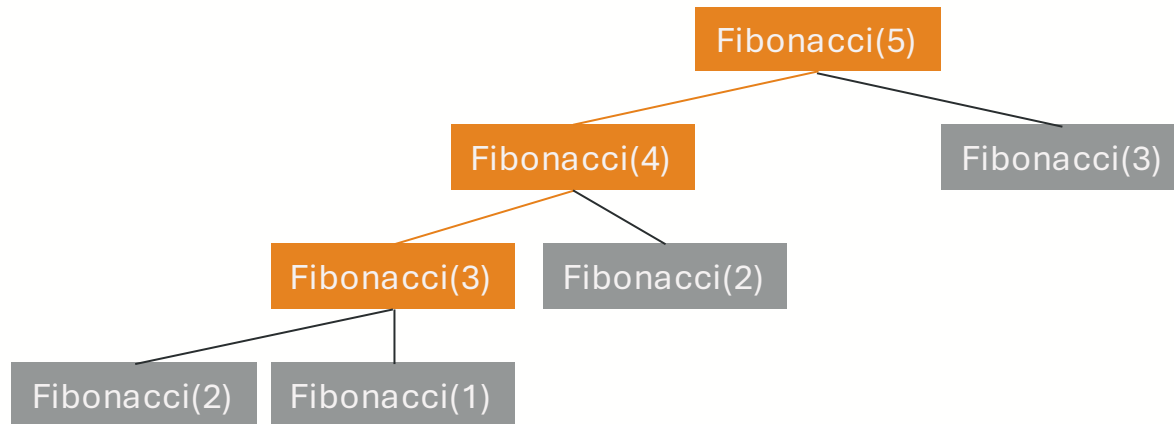
```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Fibonacci Number – Straightforward Method

- Fibonacci(5)

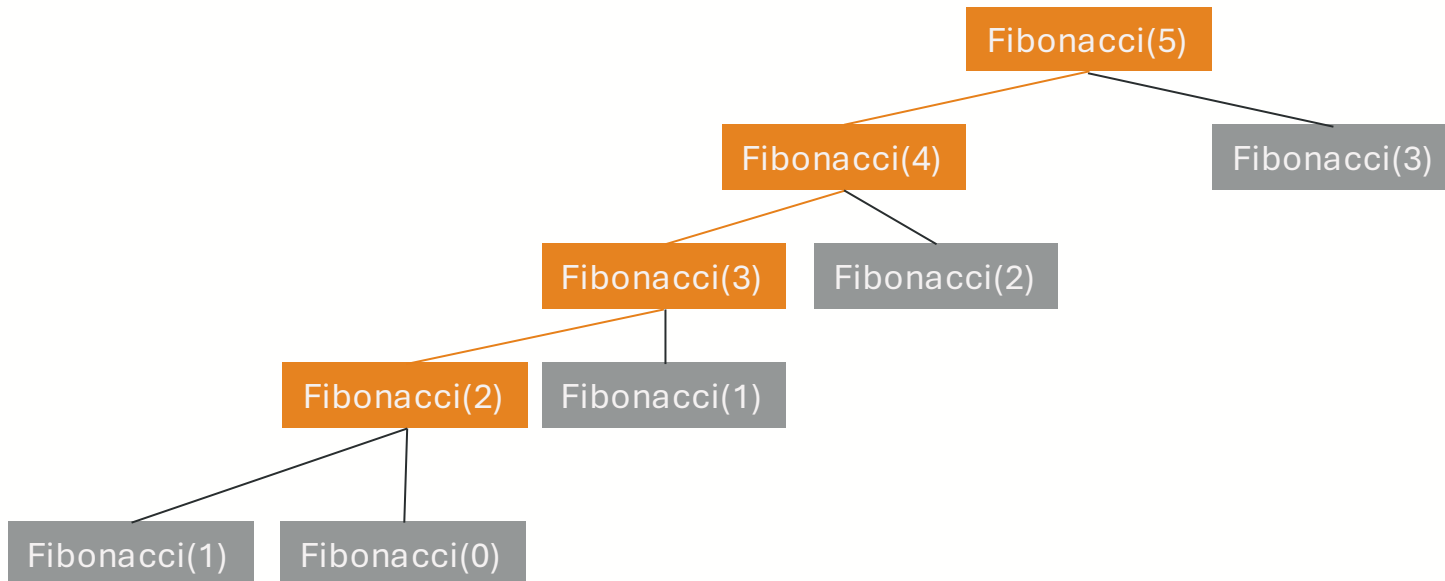
```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Fibonacci Number – Straightforward Method

- Fibonacci(5)

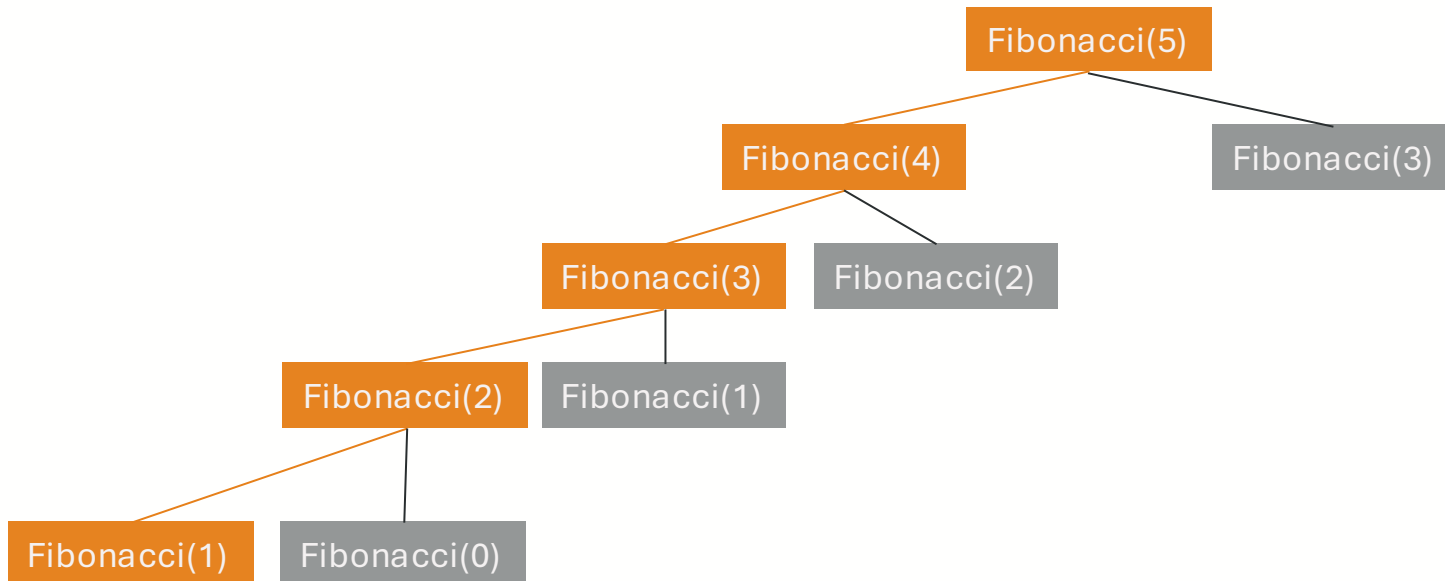
```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Fibonacci Number – Straightforward Method

- Fibonacci(5)

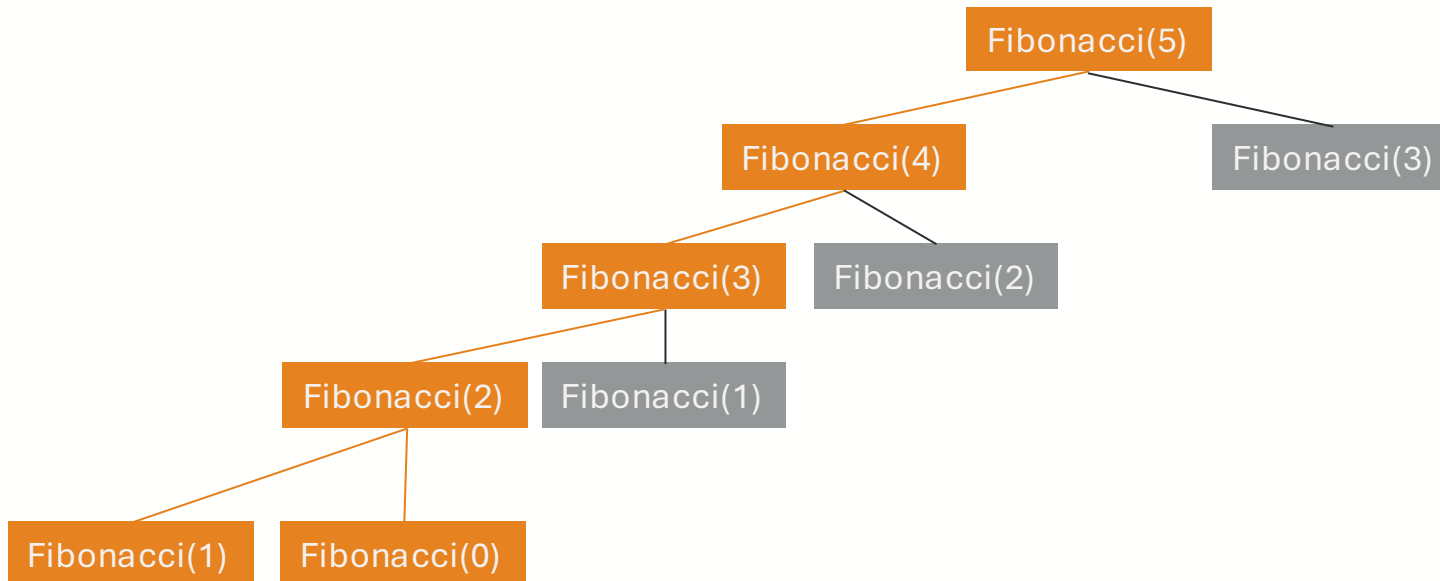
```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Fibonacci Number – Straightforward Method

- Fibonacci(5)

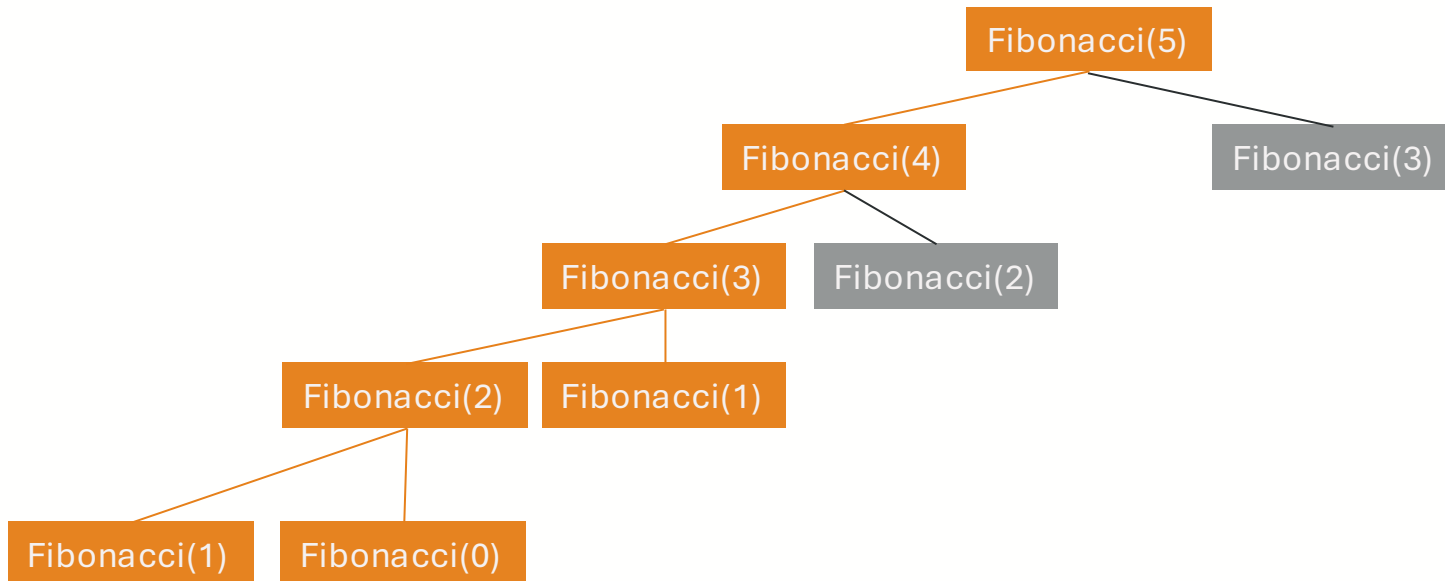
```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Fibonacci Number – Straightforward Method

- Fibonacci(5)

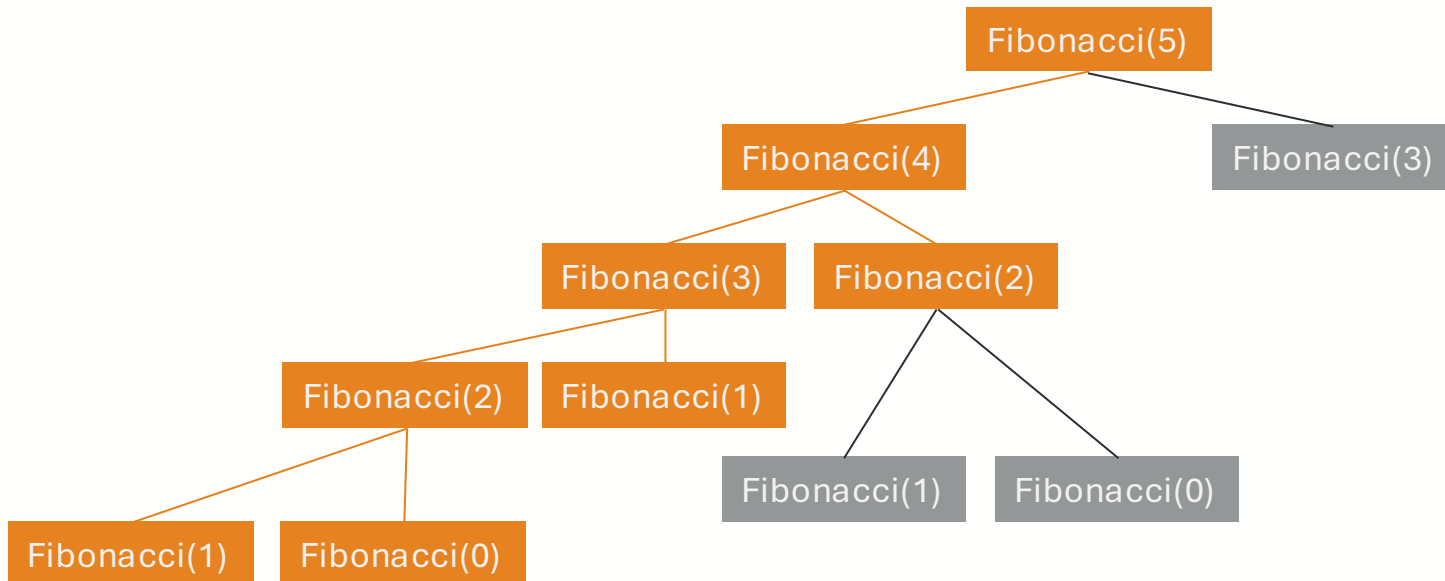
```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Fibonacci Number – Straightforward Method

- Fibonacci(5)

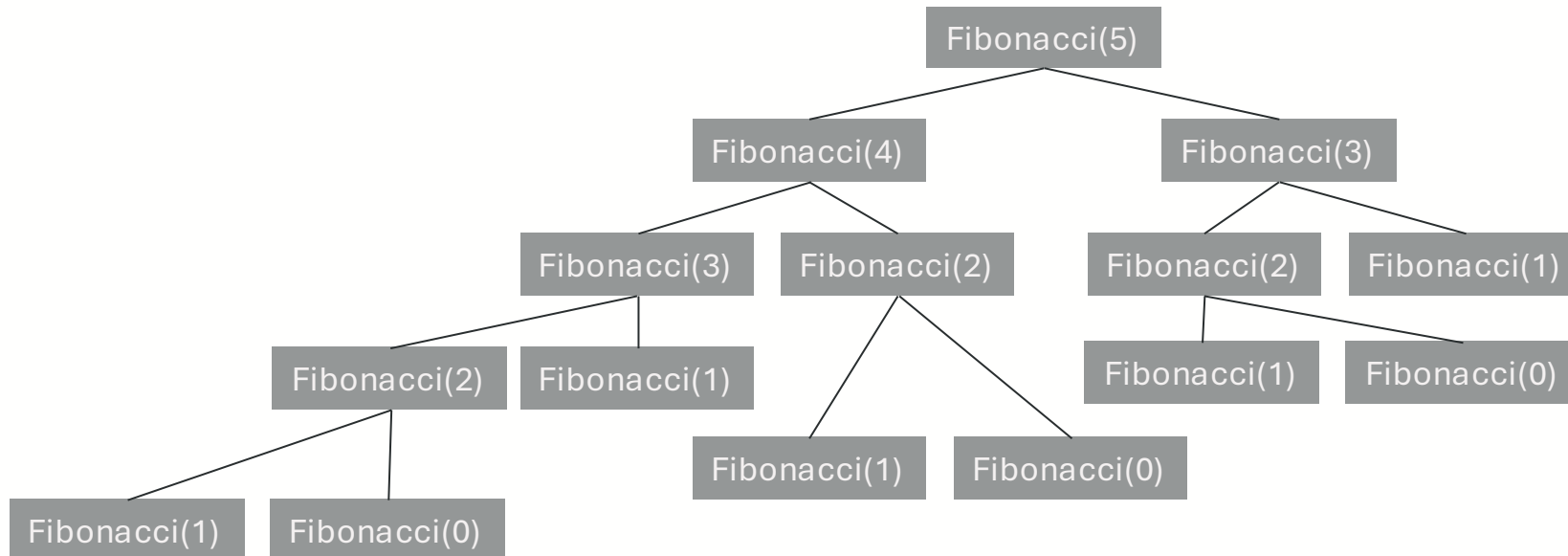
```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Fibonacci Number – Straightforward Method

- Recursion Tree for Fibonacci(5)

```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```



Time Complexity – Straightforward Method

```
function Fibonacci(n)
  if n <= 1
    return n
  else
    return Fibonacci(n-1) + Fibonacci(n-2)
```

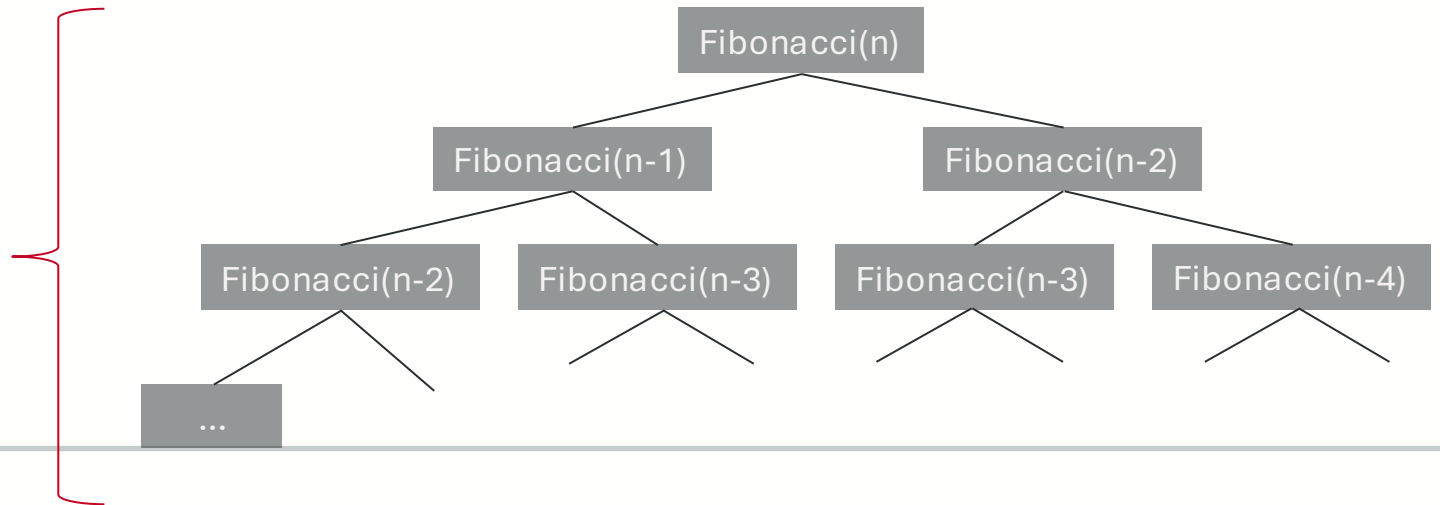
- Recurrence relation: $T(n) = T(n-1) + T(n-2) + O(1)$

- $T(n) = O(2^n)$

- Or

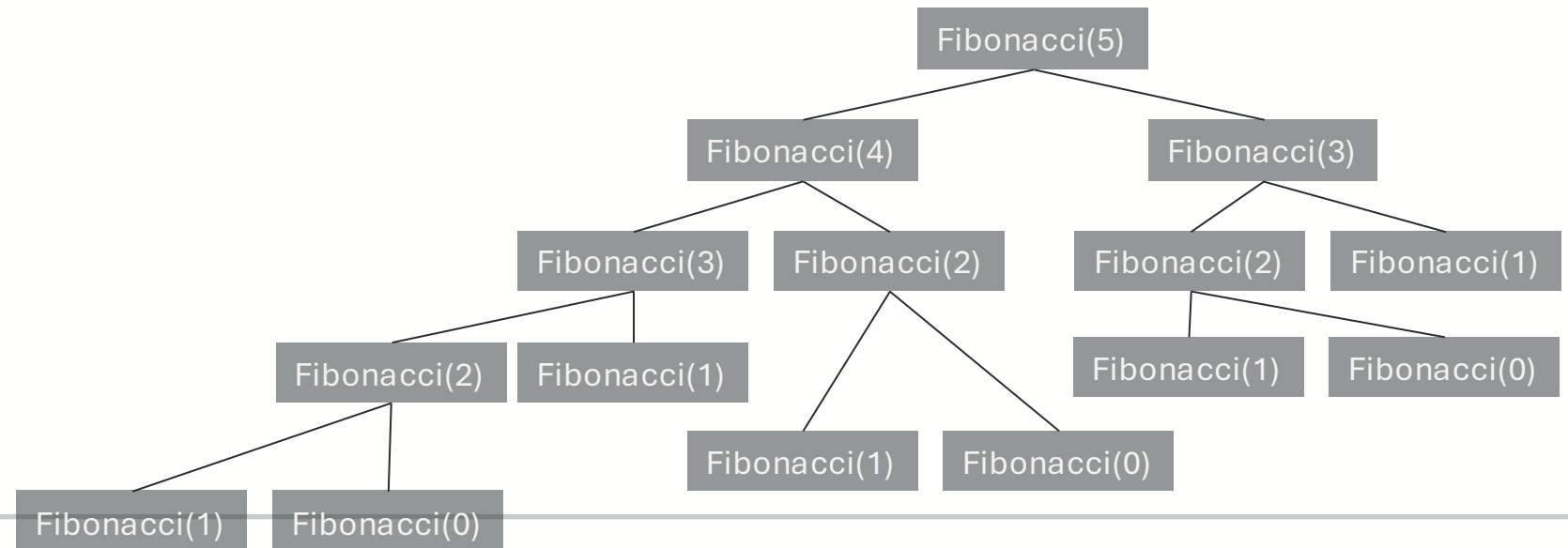
Rough Upperbound: total number of nodes in the tree:

$$\begin{aligned} &1 + 2 + 4 + \dots + 2^{n-1} \\ &= 1 \cdot \frac{2^n - 1}{2 - 1} \\ &= O(2^n) \end{aligned}$$



Avoiding Redundant Computation with DP

- DP solves subproblems only once and reuse the results to avoid duplicated computation
- What are the overlapping subproblems?
- How do we reuse these results?
 - Simple: store the results and look them up



Fibonacci Numbers – Dynamic Programming (1)

- Memoization (Top-down)
 - “Memo”, remember previously computed values

Straightforward algorithm

```
function Fibonacci(n)
    if n <= 1
        return n
    else
        return Fibonacci(n-1) + Fibonacci(n-2)
```

Memoization

```
memo={}

function Fibonacci(n)
    if n <= 1
        return n
    if n is in memo:
        return memo[n]

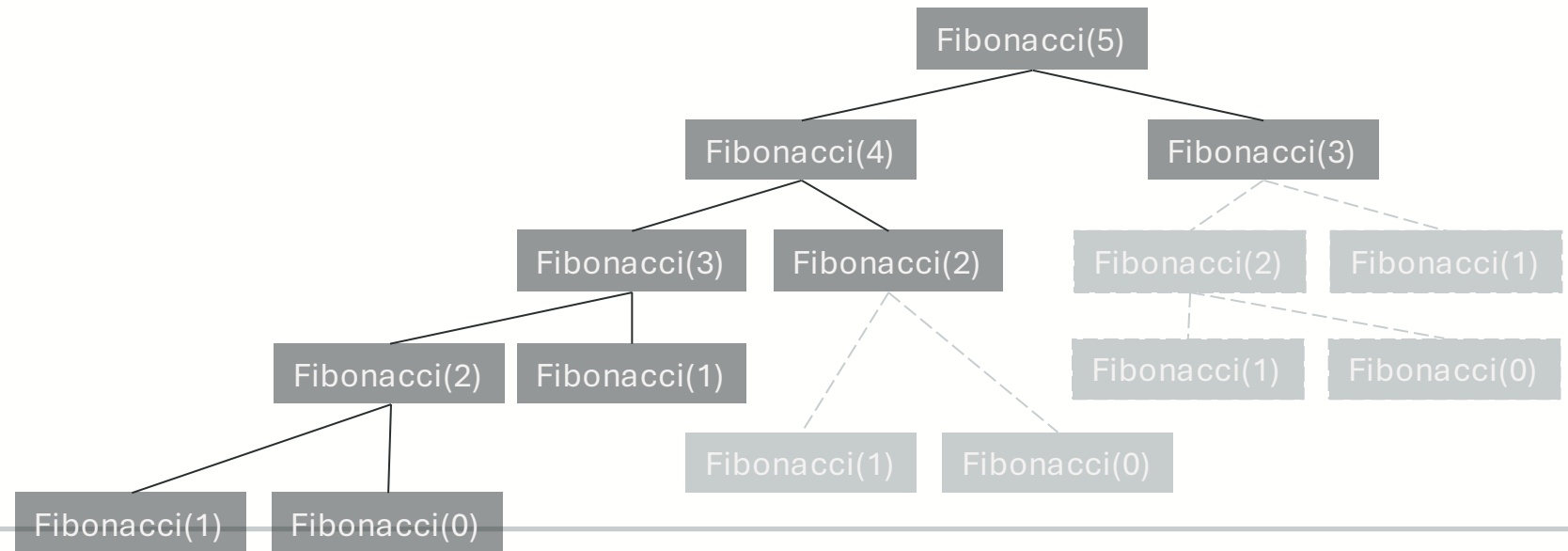
    memo[n] = Fibonacci(n-1) + Fibonacci(n-2)
    return memo[n]
```

Fibonacci Number – Memoization Example

```
memo={}  
  
function Fibonacci(n)  
    if n <= 1  
        return n  
    if n is in memo:  
        return memo[n]  
  
    memo[n] = Fibonacci(n-1) + Fibonacci(n-2)  
    return memo[n]
```

Memo

memo[2]=1
memo[3]=2
memo[4]=3
...



Memoization (Top-down) – Complexity and Considerations

- Time Complexity: $O(n)$
 - Each Fibonacci number from 2 to n is computed exactly once
- Space Complexity: $O(n)$
 - Space is required for storing the results of all subproblems
- (+) Easier to implement and understand for problems naturally expressed recursively.
- (-) Can have higher overhead due to recursive function calls
 - Can be critical when dealing with very deep recursion with limited memory

Fibonacci Numbers – Dynamic Programming (2)

- Tabulation (Bottom-up)
 - A table is used to store the results of subproblems
 - Compute $F(n)$ starting from the base cases
 - Iterative approach (e.g., using loops instead of recursion)

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	...	F_n
0	1	1	2	3	5	8	13	21	34	...	
Base cases											

$F_i = F_{i-1} + F_{i-2}, \text{ for } i \geq 2$

Fibonacci Numbers – Tabulation Method Example

```
function Fibonacci(n):  
    fib = array of size (n + 1)  
  
    fib[0] = 0  
    fib[1] = 1  
  
    for i = 2 to n:  
        fib[i] = fib[i - 1] + fib[i - 2]  
  
    return fib[n]
```

Fibonacci(5)

Initialize fib[0] = 0 and fib[1] = 1.

For i = 2: fib[2] = fib[1] + fib[0] = 1 + 0 = 1

For i = 3: fib[3] = fib[2] + fib[1] = 1 + 1 = 2

For i = 4: fib[4] = fib[3] + fib[2] = 2 + 1 = 3

For i = 5: fib[5] = fib[4] + fib[3] = 3 + 2 = 5

Table

fib[0]	0
fib[1]	1
fib[2]	1
fib[3]	2
fib[4]	3
fib[5]	5

Tabulation (Bottom-up)

- Time Complexity: $O(n)$
 - Each Fibonacci number from 2 to n is computed exactly once
- Space Complexity: $O(n)$
 - A table of size $n+1$ is used to store the Fibonacci numbers.

Summary – DP for Fibonacci Sequence Problem

- Overlapping Subproblems
 - Subproblems of $\text{Fibonacci}(n)$: $\text{Fibonacci}(n-1)$ and $\text{Fibonacci}(n-2)$
 - They often appear multiple times
- Optimal Substructure:
 - The optimal solution (the correct Fibonacci number) is built up from the optimal solutions of its subproblems
- Key Idea
 - Each Fibonacci number is computed exactly once and reused
 - Techniques:
 - Memoization (top-down)
 - Tabulation (bottom-up)

Summary – DP Strategy

- Check DP applicability
 - Optimal substructure
 - Overlapping subproblems
- Define the state
 - Choose variables that uniquely identify a subproblem
 - The state must include everything needed to make future transitions possible
- Derive the recurrence
 - Express the current state in terms of smaller states
 - Decide what the “last move” or transition was
- Add base case
 - Identify the smallest states that can be solve directly
- Compute and store
 - Top-down: recursion+memorization
 - Bottom up: iterative tabulation

Applicability to Optimization Problems

- **Optimal sub-structure (principle of optimality):** for the global problem to be solved optimally, each sub-problem should be solved optimally.
- In many problems, sub-problems overlap, meaning they are solved multiple times. This overlap can lead to redundant calculations.
- Dynamic Programming emphasizes solving every sub-problem optimally → Guarantees best solution → often at a high computational cost.
- **Small number of sub-problems:** Many NP-hard problems can be formulated as DP problems, but these formulations are not efficient, because the number of sub-problems is exponentially large.
 - The goal is to find a balance where the problem is broken down into a polynomial number of sub-problems to leverage the strengths of dynamic programming effectively.

Applicability to Optimization Problems

- While optimal sub-structure is essential for some problems, it is sometimes more efficient to be “less optimal” in solving a specific sub-problem to achieve greater overall efficiency.
- Greedy & Approximation algorithms skip exhaustive checks
 - Faster, simpler, often “good enough” for practical use.
 - Precision vs. Performance

Next Lecture

- DP for optimization problems
 - Longest Common Subsequence (LCS)
 - Matrix Chain Multiplication
- Importance of optimization in Real-World Applications
 - Efficiency: reduce costs and save resources
 - Resource allocation: optimal use of time, money and materials
 - Accurate and effective decision making

Problem 1. Climbing Stairs

- You are climbing a staircase. It takes n steps to reach the top.
- Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?
- **Constraints:** $1 \leq n \leq 45$

Example 1:

Input: $n = 2$

Output: 2

Explanation: There are two ways to climb to the top:

1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$

Output: 3

Explanation: There are three ways to climb to the top.

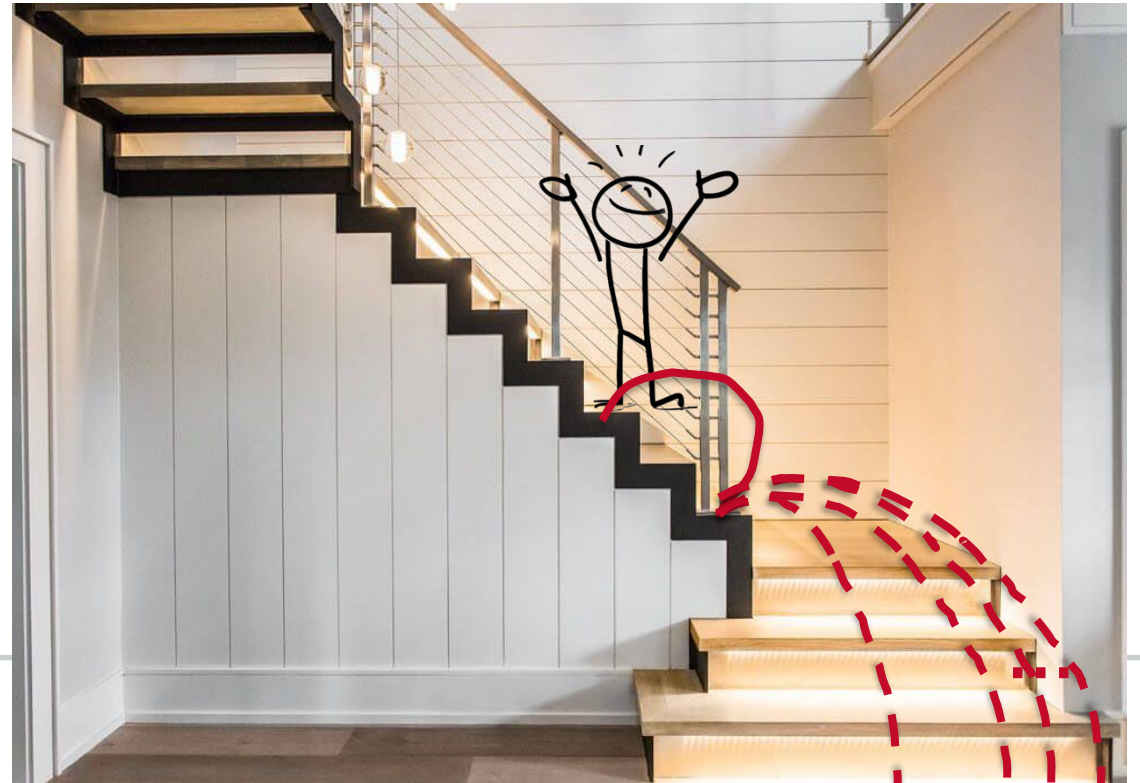
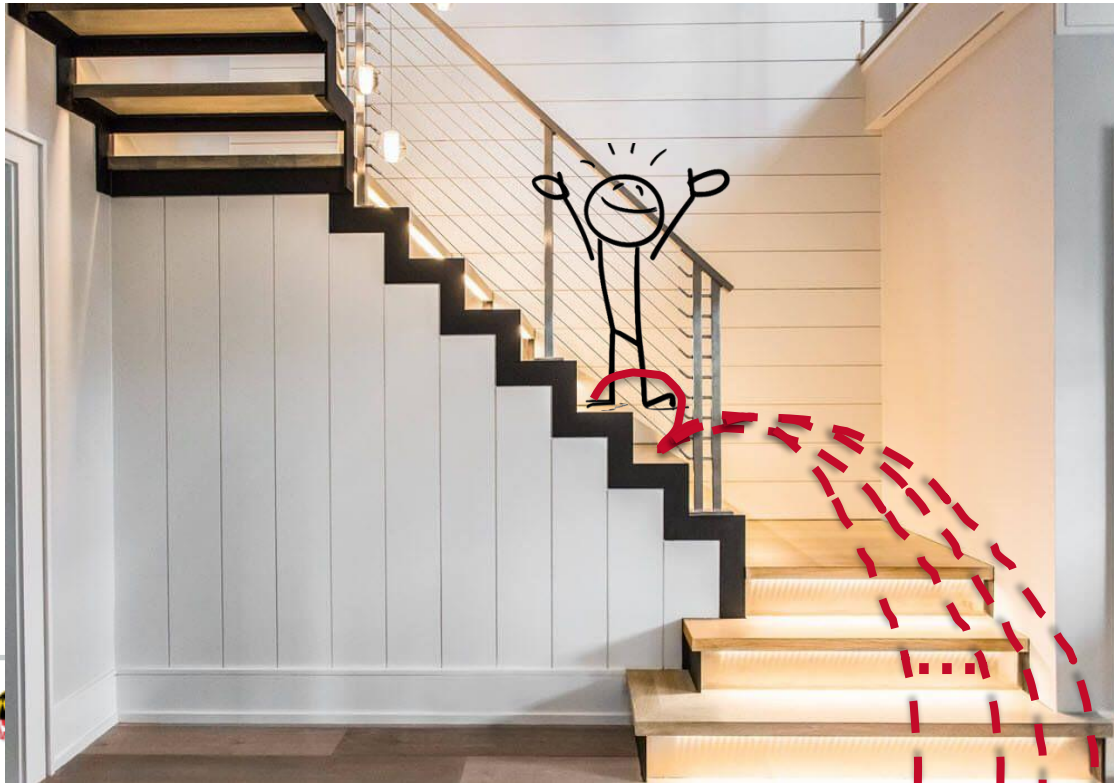
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Fill out the table

Num steps to top	0	1	2	3	4	5	6	7	8
Number of distinct ways									

Problem 1. Climbing Stairs – Optimal Substructure

- To reach step n , you must have come from either step $n-1$ or $n-2$
- Total solution for step $n = (a) + (b)$
 - (a) Solution for step $n-1$: Number of distinct ways to reach step $n-1$
 - (b) Solution for step $n-2$: Number of distinct ways to reach step $n-2$



Problem 1. Climbing Stairs – Brute Force

- Base cases
 - $\text{ClimbStairs}(1)=1$
 - $\text{ClimbStairs}(2) = 2$ ([1,1] and [2])
 - NOTE: a single base case $\text{ClimbStairs}(0)=1$ is also fine (with caution)
- Formula:
 - $\text{ClimbStairs}(n)=\text{ClimbStairs}(n-1)+\text{ClimbStairs}(n-2)$
- Brute force recursion: top-down exploration without memory
- DFS/Backtracking

Climbing Stairs - Brute force recursion

- Draw recursion tree for `ClimbStairs(6)`

Climbing Stairs – Backtracking

- Draw recursion tree for ClimbStairs(6)
- note: you can start at the target step or from 0. Label each branch with the ‘choice’ made (1 or 2 steps) and each node with the remaining steps

Problem 1. Climbing Stairs -DP

- State: $dp[i]$ =the number of ways to reach step i
- Memoization
- Tabulation
- Can we optimize the table?

Time complexity

- Brute force recursion $O(2^n)$
- DFS $O(2^n)$
- Memoization $O(n)$
- Tabulation $O(n)$

Problem 2. Coin Change

- You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.
- Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return -1.
- You may assume that you have an infinite number of each kind of coin.

Example 1:

Input: `coins = [1,2,5]`, `amount = 11`

Output: 3

Explanation: $11 = 5 + 5 + 1$

Example 2:

Input: `coins = [2]`, `amount = 3`

Output: -1

Example 3:

Input: `coins = [1]`, `amount = 0`

Output: 0

Constraints:

$$0 \leq \text{amount} \leq 1041 \leq \text{coins}[i] \\ \leq 2^{31} - 11 \leq \text{coins.length} \leq 12$$

Example 1:

Input: coins = [1,2,5]

Amount	0	1	2	3	4	5	6	7	8
Minimum number of coins									

Problem 2. Coin Change– Optimal Substructure

- The fewest coins for 11 cents using coins [1, 2, 5]
- Subproblems:
 - (a) Use 1 cent + (minimum coins for 10 cents)
 - (b) Use 2 cents + (minimum coins for 9 cents)
 - (c) Use 5 cents + (minimum coins for 6 cents)
- Optimal solution = minimum of three sub problems
$$\text{CoinChange}(n) = \min([1 + \text{CoinChange}(n-1), 1 + \text{CoinChange}(n-2), 1 + \text{CoinChange}(n-5)])$$

Problem 2. Coin Change– Optimal Substructure

- Generalize for i coins of values $[coin_1, coin_2, \dots, coin_i]$ for amount n
- $CoinChange(n) = 1 + \min([CoinChange(n - coin_1), CoinChange(n - coin_2), \dots, CoinChange(n - coin_i)])$
- Would Greedy approach work? Why or why not?

Problem 2. Coin Change – Brute Force

- Important! Focus on the State (the remaining amount), not the Choices (the coins)
- Base case: $n=0$, $\text{CoinChange}(0) \rightarrow \text{return } 0$
 - What about $\text{CoinChange}(1)$, $\text{CoinChange}(2)$, ...?
- Formula:

$\text{CoinChange}(n)$

$= 1 + \min([\text{CoinChange}(n - \text{coin}_1), \text{CoinChange}(n - \text{coin}_2), \dots, \text{CoinChange}(n - \text{coin}_i)])$

$= 1 + \min_i \{ \text{CoinChange}(n - \text{coin}_i) \mid c_i \leq n \}$

Problem 2. Coin Change - Brute force recursion

- Draw recursion tree for
 - Input: coins = [1,2,5], amount = 11

Problem 2. Coin Change - Backtracking

- Draw recursion tree for
 - Input: coins = [1,2,5], amount = 11

Problem 2. Coin Change -DP

- State: $dp[i]$ = minimum number of coins needed to make amount i
- Memoization
- Tabulation

Problem 2. Coin Change - Time complexity

- Brute force recursion: $O(k^N)$
- DFS $O(k^N)$
- Memoization $O(kN)$
- Tabulation $O(kN)$

- What about Branch and Bound?

Take-home Exercise: Implement DP solutions

- Implement both memorization (top-down) and tabulation (bottom-up) solutions for the two problems:
 - Climbing Stairs
 - Coin Change
- Thinking Questions
 - Are there subproblems that tabulation computes but memorization skips?
 - Memoization vs. tabulation. Which uses less memory in practice?
 - Can we apply greedy approach? Why and why not?
 - Can we model the problem as exploring a state space search (e.g., Backtracking, Branch and Bound)? What are the states? What are the transitions? How does DP differ from exhaustive state space search?